

Clustering

Mathematics of AI - RPTU Kaiserslautern

WiSe 25/26

Topics of the lecture

- K-means
- Selection of the number of clusters
- K-means using `scikit-learn`
- Additional clustering methods

K-means

Let us consider given data $x_t \in \mathbb{R}^D$ for $t = 1, \dots, T$ and parameter K , we aim to find K clusters with mean θ_k and binary assignments $\Gamma_{k,t}$, for each data point/cluster combination.

$$[\theta^*, \Gamma^*] = \underset{\theta, \Gamma \in \Omega_\Gamma}{\operatorname{argmin}} \sum_{k=1}^K \sum_{t=1}^T \Gamma_{k,t} \|x_t - \theta_k\|^2$$

such that: $\forall t : \sum_{k=1}^K \Gamma_{k,t} = 1, \forall k, t : \Gamma_{k,t} \in \{0, 1\}$

$\Gamma \in \{0, 1\}^{K, T}$, and $\Gamma_{k, t}$ expresses the *probability* that the t -th data point is in the k -th cluster

$\theta_k \in \mathbb{R}^D$, expresses the center of the k -th
cluster

K-means algorithm

K-means

```
1 input: data  $X$ , number of clusters  $K$ ,  
2 Set feasible initial value for  $\theta$ ,  
3 while  $L(\Gamma^{\text{it}}, \theta^{\text{it}}) - L(\Gamma^{\text{it}-1}, \theta^{\text{it}-1}) > \varepsilon$ ,  
4   Solve  $\Gamma^{\text{it}} := \operatorname{argmin}_{\Gamma \in \Omega_{\Gamma}} L(\theta^{\text{it}}, \Gamma)$ ,  
5   Solve  $\theta^{\text{it}} := \operatorname{argmin}_{\theta} L(\theta, \Gamma^{\text{it}})$ ,  
6    $\text{it} := \text{it} + 1$ ,  
7 endwhile,
```

Problem in Γ

Note that the problem is separable in t . The solution is:

$$\forall k, t : \Gamma_{k,t}^* = \begin{cases} 1 & \text{if } k = \operatorname{argmin}_{\hat{k}} \|x_t - \theta_{\hat{k}}\|^2 \\ 0 & \text{otherwise} \end{cases}$$

Problem in θ

Note that the problem is separable in k . The solution is:

$$\forall k : \theta_k^* = \frac{\sum_{t=1}^T \Gamma_{k,t} x_t}{\sum_{t=1}^T \Gamma_{k,t}}$$

Interpretation

Note that the above formulation matches the following definition:

“At every step, assign every point to the cluster with the nearest mean, then, recalculate the means. Repeat until convergence to (local) optimum.”

Random initializations

Since K-means converges to a *local* optimum, running the algorithm with different random initializations can help explore more possible solutions.

Keep this in mind in your implementation!

Selection of number of clusters

To select the number of clusters to be used, run the k-means algorithm for increasing `n_clusters` and keep track of a metric of the score (for example, the loss or the silhouette score `from sklearn.metrics import silhouette_score`).

Plot the metric for various `n_clusters` and look for a value of `K` where additional clusters lead to small improvements.

K-means using scikit-learn

The scikit-learn library offers an implementation of k-means

```
from sklearn.cluster import KMeans
```

```
m = KMeans(  
    n_clusters = 10,  
    n_init=5,  
    max_iter=400,  
    tol=1e-4,  
    ...  
) .fit(X)
```

Additional Clustering methods

`scikit-learn` offers more clustering methods. Find in the last notebook some examples. You can use different datasets, read the documentation of the methods and change the parameters.

For a more detailed comparison, see [the example on scikit-learn documentation](#)